Unsupervised Learning: Structure, Clustering, and Representations

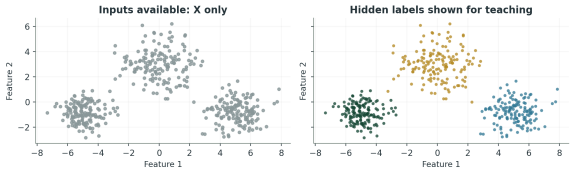
Lecture 31

CS 4880

Kirk Duran

A Dataset Can Be Valuable Without Ground-Truth Answers

In supervised learning, labels define a target; in unsupervised learning, structure in the inputs is the target.



- We observe feature vectors $x_i \in \mathbb{R}^d$ without provided class labels y_i.
- We can ask which observations resemble each other, what varies, and which are unusual.
- The colors at right are *hidden synthetic truth*, shown after the fact for teaching.

Key idea

Discovering a pattern is not the same as discovering its real-world meaning.

Three Unsupervised Learning Problems

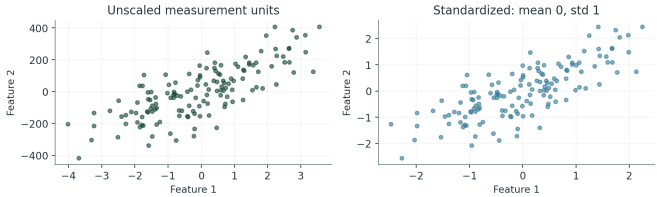
Clustering	Representations	Novelty detection
Which observations belong to similar groups? **Examples:** K-means, DBSCAN, agglomerative clustering.	Can we describe an observation with fewer, more useful coordinates? **Examples:** PCA, NMF, t-SNE.	Does a new observation resemble the training population? **Example:** One-Class SVM.

Key idea

We still need a notion of similarity, an inductive assumption, and an honest evaluation.

Even Without Labels, We Make Assumptions

Similarity depends on the representation and the units we choose.



- Raw Euclidean distance gives equal weight to a unit of temperature and a unit of distance.
- Standardize nonconstant training features to mean 0 and standard deviation 1; reuse those statistics for new inputs.
- Rescaling can change neighborhoods, centroids, and clusters.

Euclidean distance

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^{d} (x_{ir} - x_{jr})^2}.$$

Here d is the number of features; r indexes one coordinate.

How Can We Evaluate an Unsupervised Result?

- **Internal:** is a geometric objective small, or are clusters compact and separated? No class labels required.
- **External:** compare with labels reserved for analysis. Adjusted Rand index (ARI) measures agreement regardless of cluster IDs: 1 means identical partitions, around 0 means chance agreement.
- **Stability:** do reasonable changes to sampling, initialization, or hyperparameters change the answer?
- **Utility:** does the discovered representation help a later task or scientific investigation?

Key idea

A compelling two-dimensional plot is neither proof of correctness nor evidence of causation.

K-Means: Build Groups Around Representative Centers

K-means assumes a useful grouping can be described with k centroids.

1. Choose the number of groups k and initialize k centers.
2. **Assignment:** send each point to its nearest center.
3. **Update:** move each center to the mean of its assigned points.
4. Repeat until assignments or the objective stabilize.

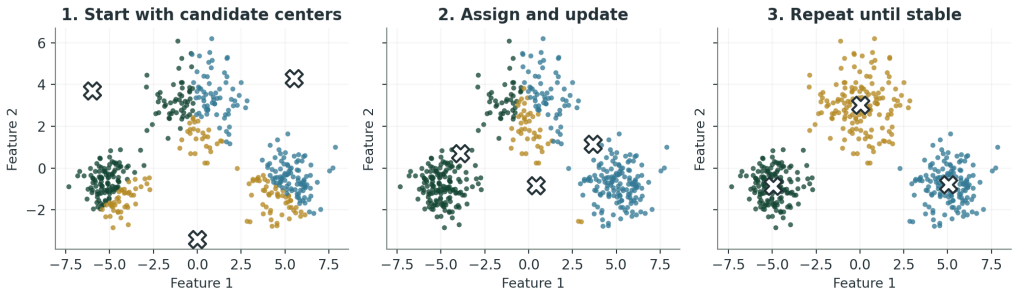
K-means objective, also called inertia

$$J = \sum_{i=1}^{n} \|x_i - \mu_{z_i}\|_2^2.$$

$z_i \in \{1, \ldots, k\}$ is the cluster assignment of point i; μ_j is centroid j. The algorithm reduces J iteratively but need not find its global minimum.

K-Means in Motion: Assign, Update, Repeat

The optimization alternates between assigning points and moving the centers.



- The large X markers denote the current centroid locations.
- Initialization matters; scikit-learn can run several starts and keep the best solution.
- A cluster ID is only a numerical index, not a semantic category such as “safe.”

Demo 1: Fit K-Means with scikit-learn

The labels are generated as a teaching check, but never passed to K-means.

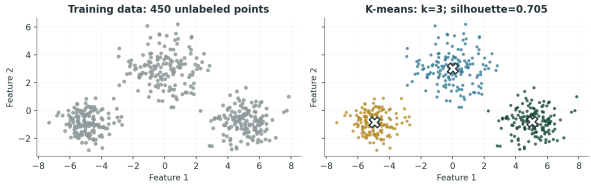
```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

X, hidden_y = make_blobs(
    n_samples=450,
    centers=[[-5,-1], [0,3], [5,-.7]],
    cluster_std=[.82, 1.17, .9], random_state=4880)
model = KMeans(n_clusters=3, n_init=10, random_state=4880)
cluster_id = model.fit_predict(X)
print(round(model.inertia_, 1))
print(round(silhouette_score(X, cluster_id), 3))
print(round(adjusted_rand_score(hidden_y, cluster_id), 3))
```

Key idea

K-means sees only X; adjusted Rand index (ARI) is a post-hoc teaching diagnostic.

Demo 1 Results: Three Groups in 450 Points



Measured run

Inertia: 897.3

Silhouette: 0.705

Post-hoc ARI: 0.987

Cluster sizes: 151, 151, 148

Interpretation: the compact, well-separated synthetic groups match K-means' geometric assumptions.

How Many Clusters? Why Inertia Alone Is Not Enough

The algorithm requires k, but the dataset does not provide a uniquely correct choice.

- More clusters almost always reduce inertia: eventually each point could be its own center.
- The **elbow** heuristic looks for diminishing improvements in inertia.
- **Silhouette** compares a point's average distance within its cluster with distance to the nearest other cluster.
- Subject-matter constraints and stability may matter more than maximizing a score.

Silhouette of an individual observation

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad \in [-1, 1].$$

$a(i)$ is average within-cluster distance; $b(i)$ is the smallest average distance to another cluster. A larger value indicates stronger separation under this distance.

Demo 2: Try Several Values of k

Run the same clustering method with different numbers of groups.

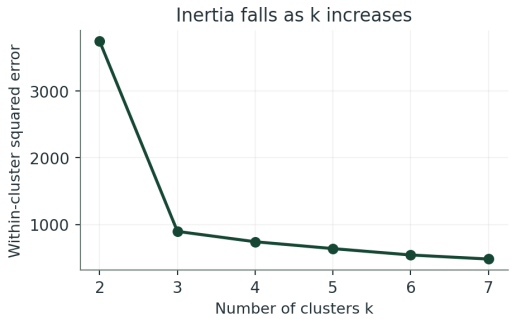
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

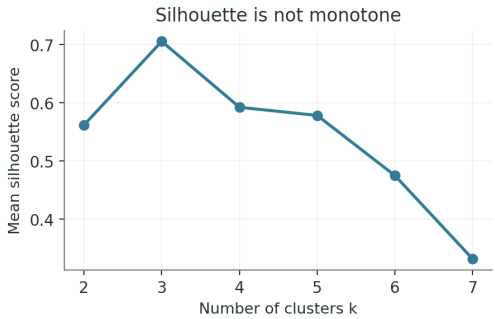
for k in range(2, 8):
    m = KMeans(n_clusters=k, n_init=10,
               random_state=4880).fit(X)
    score = silhouette_score(X, m.labels_)
    print(k, round(m.inertia_, 1), round(score, 3))
```

Printed output: k, inertia, mean silhouette

```
2:  3740.7, 0.562  3:  897.3, 0.705  4:  742.6, 0.592
5:  640.0, 0.578  6:  544.6, 0.475  7:  483.9, 0.331
```

Demo 2 Results: Inertia Versus Silhouette



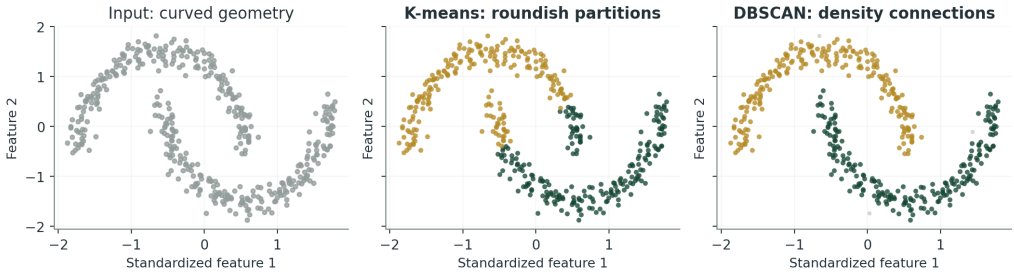


Key idea

For this dataset, the mean silhouette peaks at $k = 3$. No universal scoring rule recovers the "true" number of real-world groups.

The Geometry of the Data Can Defeat K-Means

Not all meaningful clusters look like compact blobs around a center.



- These interlocking crescents have two connected structures, not two spherical clouds.
- A centroid-based partition cuts across the curves; density connections can follow them.

DBSCAN: Define a Group by Density, Not a Center

Density-Based Spatial Clustering of Applications with Noise identifies dense, connected regions.

- ε **(eps):** the neighborhood radius around a point.
- `min_samples`**:** the minimum nearby sample count, including the point itself, for a core point.
- Core points connect through chains of dense neighborhoods; border points may join nearby groups.
- Points not assigned to a group are labeled `-1` (noise).
- It does not require choosing k in advance, but assumes a useful density scale.

Key idea

DBSCAN can discover nonconvex groups; it can struggle with differing densities or poorly selected radius.

Demo 3: K-Means Versus DBSCAN on Two Moons

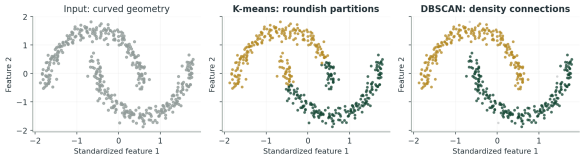
```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import adjusted_rand_score, silhouette_score

X_moon, hidden_y = make_moons(n_samples=440, noise=.07,
                              random_state=4880)
X_moon = StandardScaler().fit_transform(X_moon)
k = KMeans(n_clusters=2, n_init=10, random_state=4880)
d = DBSCAN(eps=.22, min_samples=6)
a, b = k.fit_predict(X_moon), d.fit_predict(X_moon)
print(adjusted_rand_score(hidden_y, a))
print(adjusted_rand_score(hidden_y, b))
print(silhouette_score(X_moon, a))
print(silhouette_score(X_moon, b))
```

Rounded output: K-means, then DBSCAN
Post-hoc ARI: 0.457 vs. 0.986. **Silhouette:** 0.491 vs. 0.289 (noise label included).

A Critical Result: The Better Silhouette Is Misleading

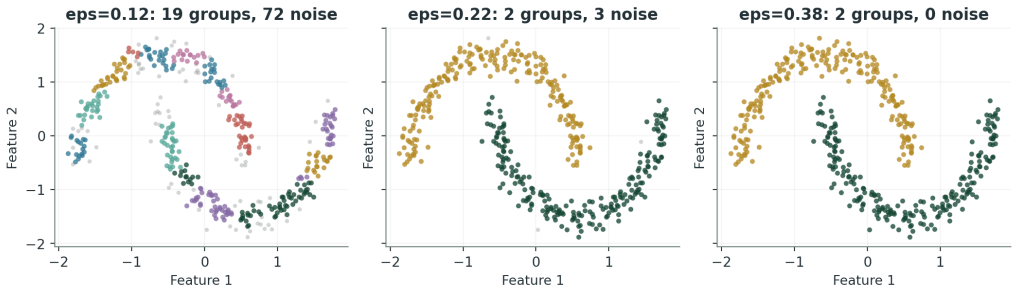
The internal score prefers K-means, while external knowledge strongly favors DBSCAN.



- **ARI** compares clusters with synthetic truth only after fitting.
- **Silhouette** rewards compact, separated partitions in the selected metric.
- Neither number tells us what the groups mean or whether they cause an outcome.

DBSCAN Is Sensitive to Its Neighborhood Scale

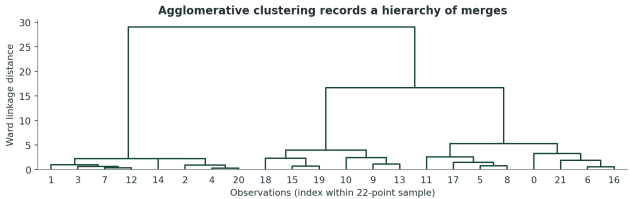
Different density thresholds can split, recover, or merge structure.



- On these standardized two-moon features, `min_samples=6` throughout.
- `eps=0.12`: 19 groups, 72 noise; `eps=0.22`: 2 groups, 3 noise; `eps=0.38`: 2 groups, 0 noise.

Agglomerative Clustering: Preserve a Hierarchy

Instead of choosing a single partition immediately, repeatedly merge groups.

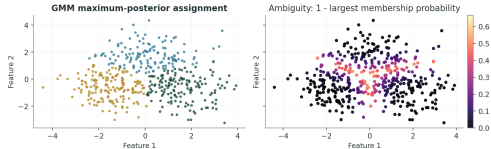


- Begin with individual points.
- Merge the best pair until one hierarchy remains.
- Different *linkage* rules define which merge is best.
- This figure uses Ward linkage on 22 sampled points.

A Probabilistic Alternative: Gaussian Mixture Models

A mixture model gives each point a degree of membership in every component.

```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
X_overlap, _ = make_blobs(
    n_samples=450,
    centers=[[-1.55,-.75],[1.5,-.65],[0,1.6]],
    cluster_std=[1.,1.,1.05],
    random_state=4880)
g = GaussianMixture(n_components=3,
    n_init=5, random_state=4880).fit(X_overlap)
prob = g.predict_proba(X_overlap)
```



Mixture density

$$p(x) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}(x \mid \mu_j, \Sigma_j), \qquad \sum_j \pi_j = 1.$$

π_j are mixing weights; μ_j and Σ_j describe each Gaussian's center and spread. Memberships are conditional on this fitted model, not guaranteed real-world classes.

When Should We Trust an Unsupervised Cluster?

- **State the intended similarity:** Euclidean distance, cosine similarity, connectivity, or a probabilistic model?
- **Inspect scale and preprocessing:** one large-unit feature can dominate distance.
- **Check alternative models:** K-means, density-based, hierarchical, and mixture assumptions differ.
- **Inspect stability:** resampling and changing seeds or parameters can reveal brittle conclusions.
- **Validate meaning:** a group of deliveries is not automatically a class called "dangerous."

Key idea

Cluster assignments are hypotheses about structure, not labels supplied by nature.

High Dimensional Does Not Always Mean High Information

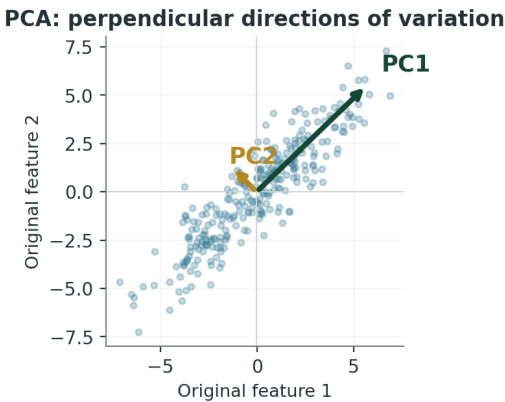
Many measured features may be redundant or correlated.



- Each scikit-learn digit is an 8×8 image, represented by 64 pixel features.
- We may seek a smaller vector $z \in \mathbb{R}^r$, $r < d$, retaining useful variation.
- A compressed representation can make visualization, storage, or later learning easier.
- Compression always introduces a tradeoff: what is retained, and what is lost?

PCA: Find Directions Along Which the Data Vary Most

Principal Component Analysis chooses new orthogonal axes, ordered by captured variance.



- PC1 is the direction with the largest variance after centering.
- PC2 is perpendicular to PC1 and explains the largest remaining variance.
- Retaining fewer axes projects data into fewer dimensions.
- The axes are computed without outcome labels.

PCA: Center, Project, and Reconstruct

Subtract the feature means, keep the strongest directions of variation, and map back to approximate the input.

First principal direction

$$v_1 = \arg\max_{\|v\|_2=1} \operatorname{Var}(X_c v), \qquad X_c = X - \mathbf{1}\mu^\top.$$

$X \in \mathbb{R}^{n\times d}$ holds n examples and d features; μ is the vector of column means; v_1 is a unit vector. Subsequent directions are orthogonal to earlier ones.

Project and approximately reconstruct

$$Z = X_c V_r, \qquad \widehat{X} = Z V_r^\top + \mathbf{1}\mu^\top.$$

$V_r \in \mathbb{R}^{d\times r}$ contains the first r principal directions; $Z \in \mathbb{R}^{n\times r}$ is the lower-dimensional representation.

- scikit-learn computes the directions numerically (SVD or a covariance-based solver); it *centers* but does not automatically standardize input features.

Demo 4: PCA on Four Measurements of Iris Flowers

We fit PCA without species labels; colors in the final plot are added after fitting.

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

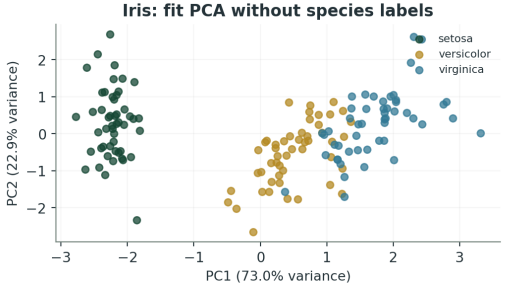
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
pca = PCA(n_components=4).fit(X)
Z = pca.transform(X)
print(pca.explained_variance_ratio_.round(4))
print(round(100 * sum(pca.explained_variance_ratio_[:2]), 1))
# Plot Z[:, 0], Z[:, 1]; colors use iris.target only AFTER fit.
```

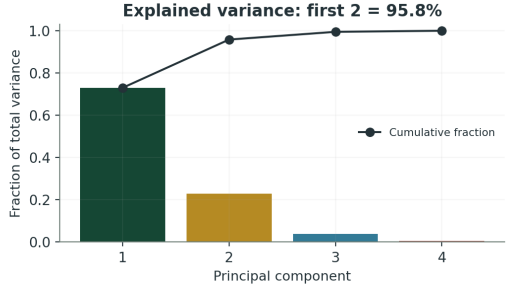
Rounded output

```
[0.7296, 0.2285, 0.0367, 0.0052]
```

First 2 PCs: **95.8%** of standardized-feature variance.

Demo 4 Results: A Two-Dimensional Iris Representation





Key idea

Two coordinates preserve 95.8% of total variance *in the standardized four-feature dataset*. Variance retained is not the same as label accuracy or physical meaning.

Explained Variance Helps Choose the Dimension

A lower-dimensional representation retains a fraction of the original variability.

- **Explained variance ratio** $\lambda_j / \sum_\ell \lambda_\ell$ measures each component's fraction of total variance.
- **Cumulative explained variance** adds those fractions for the retained components.
- Choosing r based on 90%, 95%, or another threshold is a design decision, not a guarantee of task performance.
- PCA can miss nonlinear geometry or a low-variance feature that is important for a prediction task.

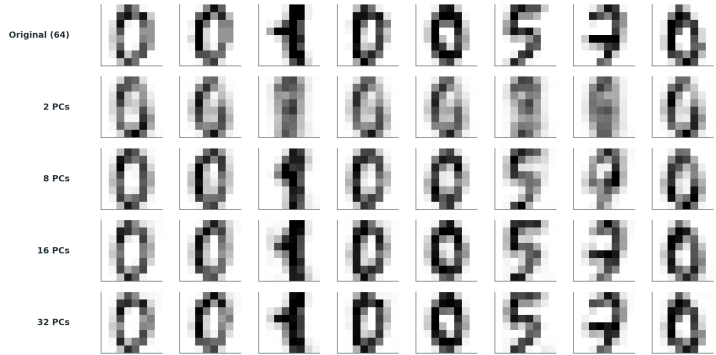
Iris, standardized first two components

$$0.7296 + 0.2285 \approx 0.9581.$$

The approximation agrees with the plotted **95.8%**. The remaining two components account for about 4.2%.

PCA Also Defines a Reconstruction Experiment

To ask what information the representation preserves, reconstruct the original signal.



- More components improve reconstruction, but do not guarantee human recognizability or classifier accuracy.

Demo 5: Compress and Reconstruct Handwritten Digits

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error

digits = load_digits()
X = digits.data                  # 1797 x 64, no labels needed
for r in [2, 8, 16, 32]:
    pca = PCA(n_components=r, svd_solver='full').fit(X)
    compressed = pca.transform(X)
    reconstruction = pca.inverse_transform(compressed)
    print(r, round(mean_squared_error(X, reconstruction), 2))
```

Measured reconstruction mean squared error, in pixel-value units squared

```
2 PCs:  13.42  8 PCs:  6.12  16 PCs:  2.83  32 PCs:  0.63
```

Key idea

This is *in-sample* reconstruction, not held-out classification or evidence of semantic understanding.

Representations Can Be Learned in Other Ways

PCA is only one way to define a useful lower-dimensional code.

- **PCA:** signed, orthogonal linear components maximize retained variance.
- **Nonnegative Matrix Factorization (NMF):** when input values are nonnegative, learn nonnegative additive basis patterns.
- **Autoencoders:** neural networks learn an encoder (input to code) and decoder (code to reconstruction); the reconstruction loss trains both. Lecture 30 introduced nonlinear architectures.
- **Task-specific representation:** features may be useful because they help downstream prediction, not simply because they reconstruct well.

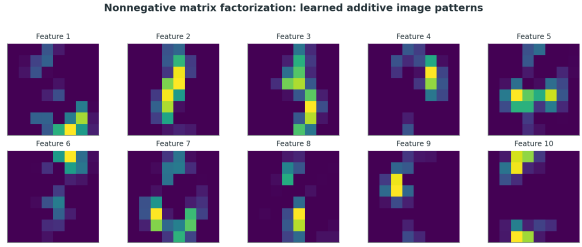
NMF factorization

$$X \approx WH, \qquad W \geq 0,\ H \geq 0.$$

$W \in \mathbb{R}^{n \times r}$ gives example coefficients; $H \in \mathbb{R}^{r \times d}$ contains learned additive basis vectors.

Demo 6: Learn Additive Digit Patterns with NMF

```
from sklearn.decomposition import NMF
from sklearn.datasets import load_digits
import numpy as np
X = load_digits().data   # all >= 0
m = NMF(n_components=10,
        init='nndsvda',
        random_state=4880,
        max_iter=1200)
W = m.fit_transform(X)
H = m.components_
relative_error = (
    np.linalg.norm(X - W @ H)
    / np.linalg.norm(X))
print(round(relative_error, 3))
```



Key idea

Measured relative Frobenius error: **0.326**. Components are learned image patterns, not necessarily digit classes or human-interpretable parts.

t-SNE: A Nonlinear Visualization of Neighborhoods

t-distributed Stochastic Neighbor Embedding seeks a low-dimensional map that preserves local similarities.

- PCA maximizes global variance along linear directions.
- t-SNE instead tries to place near-neighbors in the original space close together in the map.
- It uses an optimization objective based on probability distributions of pairwise similarities.
- **Perplexity** changes the effective neighborhood scale; initialization and random seed can change the picture.
- It is primarily a visualization method; distances *between* separated map groups can be misleading.

Key idea

Seeing discrete islands in a t-SNE figure does not prove that the data contain those many natural clusters.

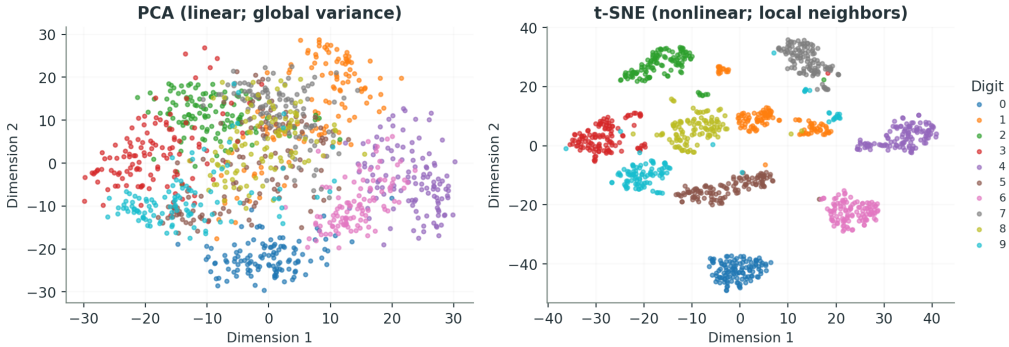
Demo 7: Visualize the Same Digits with PCA and t-SNE

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
D = load_digits()
idx = np.random.default_rng(4880).choice(
    len(D.data), size=1150, replace=False)
X = D.data[idx]
X40 = PCA(n_components=40, random_state=4880).fit_transform(X)
Z_pca = X40[:, :2]
Z_tsne = TSNE(n_components=2, perplexity=30,
              learning_rate='auto', init='pca',
              max_iter=650, random_state=4880).fit_transform(X40)
# Plot both; use D.target[idx] ONLY to color points afterward.
```

Key idea

Both embeddings learn from image features alone, not the digit identities.

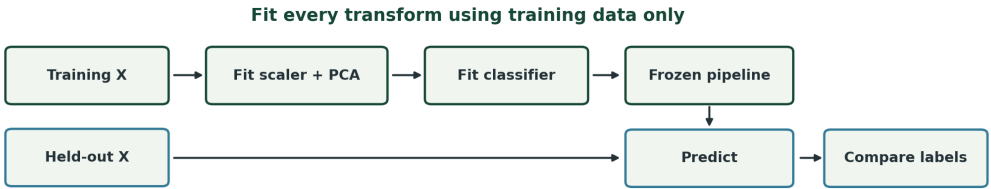
Demo 7 Results: Same Inputs, Different Geometry



- Digit labels were used solely to color the points *after* embedding.
- Global distances and cluster areas in the t-SNE map should not be read as original-space distances or densities.

A Good Representation Must Generalize Beyond Its Plot

Evaluate a transformation on data it did not help fit.



- Fit scalers, PCA, and any learned transformation on the **training split**.
- Apply the frozen transformation to validation or test observations.
- Choose representation size using validation data, then retrain on development data.
- Test once after the choices are fixed; otherwise the test set silently becomes tuning data.

Demo 8: PCA as an Unsupervised Step in a Supervised Pipeline

Fit PCA without labels; evaluate its features using a supervised classifier.

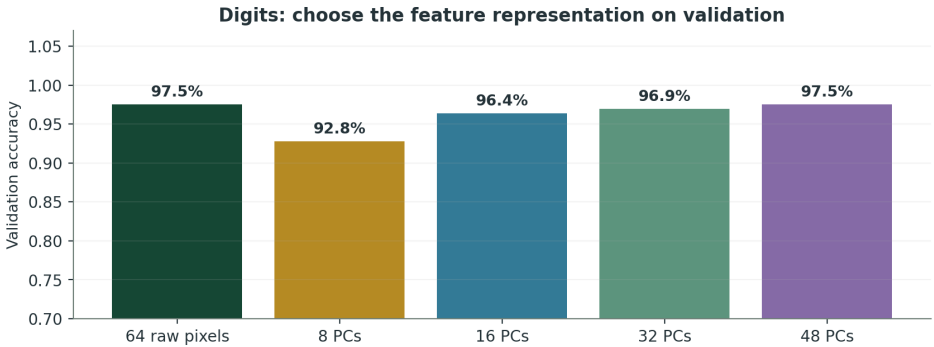
```
from sklearn.datasets import load_digits
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X, y = load_digits(return_X_y=True)
Xdev, Xtest, ydev, ytest = train_test_split(
    X, y, test_size=.20, stratify=y, random_state=4880)
Xtr, Xval, ytr, yval = train_test_split(
    Xdev, ydev, test_size=.25, stratify=ydev, random_state=4880)
def candidate(components):
    stages = [StandardScaler()]
    if components is not None:
        stages += [PCA(n_components=components, random_state=4880)]
    stages += [SVC(C=5, gamma='scale', kernel='rbf')]
    return make_pipeline(*stages)
```

Demo 8: Select on Validation, Then Test Once

```
choices = [None, 8, 16, 32, 48]   # None => original 64 pixels
validation = {}
for r in choices:
    model = candidate(r).fit(Xtr, ytr)
    validation[r] = accuracy_score(yval, model.predict(Xval))
best_r = max(choices, key=lambda r: validation[r])
final = candidate(best_r).fit(Xdev, ydev)
print(best_r, round(accuracy_score(ytest,
                                   final.predict(Xtest)), 4))
```

Measured split: 1077 train, 360 validation, 360 test
Best validation score is tied by **raw 64 pixels** and **48 PCs**, each at **97.5%**. The tie breaks toward the first candidate: **raw pixels**. After retraining, **final test accuracy = 97.78%**.

Demo 8 Results: PCA Does Not Always Help Prediction



- PCA reduces features but can discard information a classifier uses.
- Validation selects an architecture; the test score is only reported after selection and refitting.
- We should measure runtime, storage, and stability if compression is the objective.

Where Does SVM Fit? Standard SVM Is Supervised

An ordinary support vector classifier requires class labels. One-Class SVM learns an inlier region.

`sklearn.svm.SVC`	`sklearn.svm.OneClassSVM`
Supervised classification. Fit pairs (X, y); seek a wide separating margin, with a kernel allowing nonlinear boundaries. Used for digits in Demo 8.	**Novelty detection.** Fit examples assumed to represent normal operation; estimate a boundary around their support.

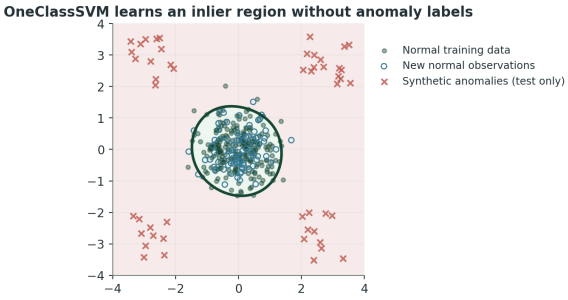
- This distinction matters: a One-Class SVM does *not* magically label multiple unknown classes.
- It tests resemblance under a kernel and scale. Predictions are $+1$ (inlier) or -1 (outlier); ν controls the training-error/support-vector tradeoff, not a guaranteed future anomaly rate.

Demo 9: Fit a One-Class SVM to Normal Data

```
import numpy as np
from sklearn.svm import OneClassSVM
rng = np.random.default_rng(4880)
normal_train = rng.normal(0, .6, size=(240, 2))
normal_test = rng.normal(0, .6, size=(80, 2))
anomaly_test = rng.uniform(2, 3.6, size=(50, 2)) * \
               rng.choice([-1, 1], size=(50, 2))
m = OneClassSVM(kernel='rbf', gamma=.4, nu=.05)
m.fit(normal_train)
print((m.predict(normal_test) == 1).mean())
print((m.predict(anomaly_test) == -1).sum())
```

Measured output
Normal acceptance: 0.925 (74 of 80). **Synthetic anomalies flagged:** 50 of 50.

Demo 9 Results: Novelty Is Not a Diagnosis



- The green contour is the learned *decision boundary* around training inliers.
- Outliers were deliberately placed far away; this is an *easy synthetic test*.
- "Outside normal" is not proof of an equipment fault, fraud, or danger.
- Real systems must measure false alarms under deployment shift.

Match the Question to the Unsupervised Method

Method	What it looks for	Main assumption / caveat
K-means	k centroid-based partitions	Compact groups; choose k; sensitive to scaling
DBSCAN	Connected dense regions	Choose density scale; differing densities are hard
Agglomerative	Nested merges / hierarchy	Linkage changes the hierarchy
Gaussian mixture	Soft component memberships	Model family and number of components matter
PCA	Low-rank linear projection	Maximizes variance, not predictive value
NMF	Additive nonnegative factors	Requires $X \geq 0$; factors need not be semantic
t-SNE	Low-dimensional neighbor visualization	Plot geometry is not a reliable global metric
One-Class SVM	Boundary of familiar observations	Sensitive to kernel/scale and training purity

Key idea

Choose the method for the scientific or operational question, not for the most visually striking figure.

A Practical Checklist Before Claiming Discovery

1. Define which variables exist and which labels, if any, are missing.
2. Document units, scaling, distance metric, randomness, and model assumptions.
3. Separate fitting from evaluation; do not let future or test data alter preprocessing.
4. Compare plausible alternatives and assess parameter sensitivity.
5. Distinguish geometric scores, post-hoc external labels, and actual task utility.
6. Have a domain expert inspect whether the representation is operationally meaningful.

Key idea

Unsupervised learning finds structure in observations. Humans must still test what that structure means.

Connecting Lecture 31 Back to Learning Agents

A delivery robot can collect far more sensor history than people can manually label.

- **Clustering:** organize similar traversal segments for human review.
- **Dimensionality reduction:** compress correlated sensor features into a smaller learned state description.
- **Anomaly detection:** flag an unusual motor or navigation signature for inspection.
- **Supervised follow-up:** once humans label some examples, evaluate whether the discovered features predict real outcomes.

Key idea

Do not treat a cluster ID as a safety label or a novelty flag as a confirmed mechanical failure.

References, Implementation, and Reproducibility

- scikit-learn User Guide: Clustering; Matrix decomposition and PCA; Manifold learning; Novelty and outlier detection.
- scikit-learn OneClassSVM and Pipeline API documentation.
- MacQueen (1967), *Some Methods for Classification and Analysis of Multivariate Observations*; Ester et al. (1996), *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*.
- Pearson (1901), *On Lines and Planes of Closest Fit to Systems of Points in Space*; van der Maaten and Hinton (2008), *Visualizing Data using t-SNE*; Schölkopf et al. (2001), *Estimating the Support of a High-Dimensional Distribution*.
- Reproducibility: the companion `generate-figures.py` regenerates all 18 included figures and numerical outputs from built-in scikit-learn datasets and synthetic data; no web access required. Verified using scikit-learn 1.8.0, NumPy 2.3.5, Matplotlib 3.10.8, and SciPy 1.18.1; dependencies are pinned beside the script.
- All numeric outcomes are *measured on the supplied run*; different library versions or environments can produce slightly different fits or embeddings. Figures with ground-truth coloring never used that coloring to fit an unsupervised model.